Soluciones – Práctica: scripts con sed

ASO · 2º ASIR · Oct 19, 2026 · @Angela

Preparación

Los cinco scripts se han probado con GNU sed 4.9 en ~/lab-sed. Antes de empezar, el alumnado crea la carpeta, copia los ficheros y genera los dos ficheros de prueba:

```
mkdir ~/lab-sed
cd ~/lab-sed
cp /etc/login.defs /etc/passwd .

printf
'jgarcia@gmail.com\nlucia.ferrer@gmail.com\nhugo.marti@hotmail.com\nnerea.sol
er@gmail.com\n' > correos.txt

cat > accesos.log <<'EOF'
Oct 12 08:01:12 srv sshd[1201]: Failed password for root from 185.220.101.4
port 51234 ssh2
Oct 12 08:02:40 srv sshd[1210]: Accepted publickey for jgarcia from
192.168.10.15 port 50122 ssh2
Oct 12 08:07:44 srv sshd[1322]: Failed password for invalid user admin from
45.95.147.9 port 40022 ssh2
EOF

chmod +x *.sh
```

Ejercicio 1. Configuración limpia

```
#!/bin/bash
# config_limpia.sh - Muestra un fichero de configuración sin comentarios ni
líneas vacías

if [ $# -ne 1 ]; then
    echo "Uso: $0 fichero"
    exit 1
fi

if [ ! -f "$1" ]; then
    echo "Error: el fichero $1 no existe"
    exit 1
fi

sed -E '/^(#|$)/d' "$1"
```

- La `d` borra las líneas que cumplen el patrón; sed muestra el resto. Sin `-i`, el fichero no se modifica.

Otras soluciones válidas

```
sed -E '/^(#.*)?$/d' "$1"

sed '/^#/d; /^$/d' "$1"
```

Ejercicio 2. Cambiar el dominio de los correos

Una sola sustitución con alternancia `(gmail|hotmail)` cubre los dos dominios.

```
#!/bin/bash
# cambiar_dominio.sh - Cambia @gmail.com y @hotmail.com por @alu.edu.gva.es

if [ $# -ne 1 ]; then
    echo "Uso: $0 fichero"
    exit 1
fi

if [ ! -f "$1" ]; then
    echo "Error: el fichero $1 no existe"
    exit 1
fi

sed -E 's/@(gmail|hotmail)\.com$/@alu.edu.gva.es/' "$1"
```

- `E`n la parte de reemplazo los puntos no tienen significado especial, así que `@alu.edu.gva.es` se escribe tal cual.

Ejercicio 3. Anonimizar un registro

La clave es redirigir la salida de sed a un fichero nuevo en lugar de usar `-i`, para que el original quede intacto.

```
#!/bin/bash
# anonimizar_log.sh - Genera una copia del registro con las IP sustituidas
por X.X.X.X

if [ $# -ne 1 ]; then
    echo "Uso: $0 fichero"
    exit 1
fi

if [ ! -f "$1" ]; then
    echo "Error: el fichero $1 no existe"
    exit 1
fi

salida="$1.anonimo"

sed -E 's/[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}/X.X.X.X/g' "$1" >
"$salida"

echo "Fichero anonimizado: $salida"
```

- La `g` final sustituye todas las IP de cada línea, no solo la primera.

- La expresión acepta valores imposibles como 999.999.999.999. Para anonimizar es suficiente; validar que cada bloque vaya de 0 a 255 complicaría mucho la expresión.

Otras soluciones válidas

```
sed -E 's/([0-9]{1,3}\.){3}[0-9]{1,3}/X.X.X.X/g' "$1" > "$salida"

sed -E 's/from [0-9.]+/from X.X.X.X/' "$1" > "$salida"
```

Ejercicio 4. Caducidad de las contraseñas

login.defs contiene PASS_MAX_DAYS dos veces: en un comentario y en la línea real. Hay que anclar con `^` tanto el grep como el sed para tocar solo la línea real.

```
#!/bin/bash
# caducidad.sh - Cambia PASS_MAX_DAYS en la copia login.defs

fichero=login.defs

if [ $# -ne 1 ]; then
    echo "Uso: $0 dias"
    exit 1
fi

if [[ ! "$1" =~ ^[0-9]{1,3}$ ]]; then
    echo "Error: $1 no es un número de días válido"
    exit 1
fi

antes=$(grep -E '^PASS_MAX_DAYS' "$fichero")

sed -i.bak -E "/^PASS_MAX_DAYS/s/[0-9]+/$1/" "$fichero"

despues=$(grep -E '^PASS_MAX_DAYS' "$fichero")

echo "Antes: $antes"
echo "Después: $despues"
```

- `^[0-9]{1,3}$` exige de 1 a 3 cifras y nada más: rechaza `noventa`, `1000` y `-5`.
- `-i.bak` modifica el fichero y guarda el original como login.defs.bak. Va sin espacio: `-i .bak` no funciona.

- `/^PASS_MAX_DAYS/` delante de la `s` es una dirección: la sustitución solo se aplica a la línea que empieza por PASS_MAX_DAYS, y en ella se cambia el número. El tabulador original se conserva porque no se toca.
- El sed va entre comillas dobles para que Bash sustituya `$1`. Con comillas simples aparecería el texto literal $1.
- Alternativa también correcta: `s/^PASS_MAX_DAYS.*/PASS_MAX_DAYS\t$1/`, que reescribe la línea entera.

Ejercicio 5. Cambiar la shell de un usuario

El sed combina una dirección (solo la línea que empieza por `usuario:`) con una sustitución del último campo.

```
#!/bin/bash
# cambiar_shell.sh - Cambia la shell de un usuario en la copia passwd

fichero=passwd

if [ $# -ne 2 ]; then
    echo "Uso: $0 usuario shell"
    exit 1
fi

usuario=$1
shell=$2

if ! grep -q "^$usuario:" "$fichero"; then
    echo "Error: el usuario $usuario no existe"
    exit 1
fi

if [[ ! "$shell" =~ ^/bin/(bash|sh|zsh)$ ]]; then
    echo "Error: la shell debe ser /bin/bash, /bin/sh o /bin/zsh"
    exit 1
fi

sed -i.bak "/^$usuario:/s|:[^:]*$|:$shell|" "$fichero"

grep "^$usuario:" "$fichero"

# Pregunta: en un sistema real, ¿se cambiaría la shell editando /etc/passwd
con sed?
# No. Editar /etc/passwd a mano es arriesgado: un error de sintaxis puede
dejar
```

```
# usuarios (incluido root) sin poder iniciar sesión, y no se hace ninguna
comprobación.
# Lo normal es usar:
#   sudo usermod -s /bin/zsh ubuntu   (el administrador, para cualquier
usuario)
#   chsh -s /bin/zsh                  (el propio usuario, para sí mismo)
# Ambos comprueban que la shell es válida (está en /etc/shells) y bloquean el
# fichero mientras lo modifican.
```

- `"^$usuario:"` incluye los dos puntos: sin ellos, buscar ubuntu también encontraría un usuario ubuntu2.
- `/^$usuario:/` delante de la `s` es una dirección: la sustitución solo se aplica a esa línea.
- `:[^:]*$` es el último campo: dos puntos seguidos de caracteres que no son dos puntos hasta el final de línea.
- Se usa `|` como delimitador de la `s` porque la shell contiene barras. Con `/` habría que escaparlas: `s/:[^:]*$/:\/bin\/zsh/`.
- `^/bin/(bash|sh|zsh)$` anclada por ambos lados rechaza `/bin/fish` y también `/bin/bashx` o `/usr/bin/bash`.